Rethinking ABM Architecture

Entity Component Systems and the Case for Parallel Agent Interaction

Franz Scharnreitner

ICAE Linz

2026-03-04

1. Cars driving in circles

1. Cars driving in circles

—

1.1 The original model

- Influential paper by Hodgson (2006)

1. Cars driving in circles
— 1.1 The original model

1.1 The original model

- Influential paper by Hodgson (2006)

1. Cars driving in circles
— 1.1 The original model

1.2 Overview

- Cars move sequentially, either clockwise or counterclockwise, on a 100×2 ring.
- In each step, a car chooses the left or right lane according to the decision formula

$$\mathrm{LR}^n = s^n + o^n + c^n + h^n$$

.

- Here, s^n, o^n, and c^n depend on the cars ahead of car n, while h^n represents the driver's habit.
- Driving on one lane shifts habit toward that lane:

1. Cars driving in circles
— 1.2 Overview

1.2 Overview

$$h^n = h^n_{\text{prev}} + \frac{\text{lane}}{K + t^n}$$

.

1. Cars driving in circles
— 1.2 Overview

2. Main results of Hodgson & Knudsen

2. Main results of Hodgson & Knudsen

- **Habit is crucial for the emergence of convention.**
- **Repeated lane choices gradually become**

2. Main results of Hodgson & Knudsen

—

ingrained, making left/right coordination more stable.

- Agents do not coordinate only because

2. Main results of Hodgson & Knudsen

—

of immediate incentives; they also coordinate because past behavior shapes current disposition.

- **Habit therefore strengthens and stabilizes conventions beyond what purely**

reactive decision-making can achieve.

- The paper's broader claim is that institutions persist

2. Main results of Hodgson & Knudsen

—

partly because they become internalized as habits.

2.1 The real world

- Real-world agents, especially drivers, act in parallel.
- So why was the original model implemented sequentially?
  - Sequential updating is simple.
  - It is the most natural update scheme in many traditional ABM frameworks.
  - But this also reflects the underlying software architecture.

2. Main results of Hodgson & Knudsen
— 2.1 The real world

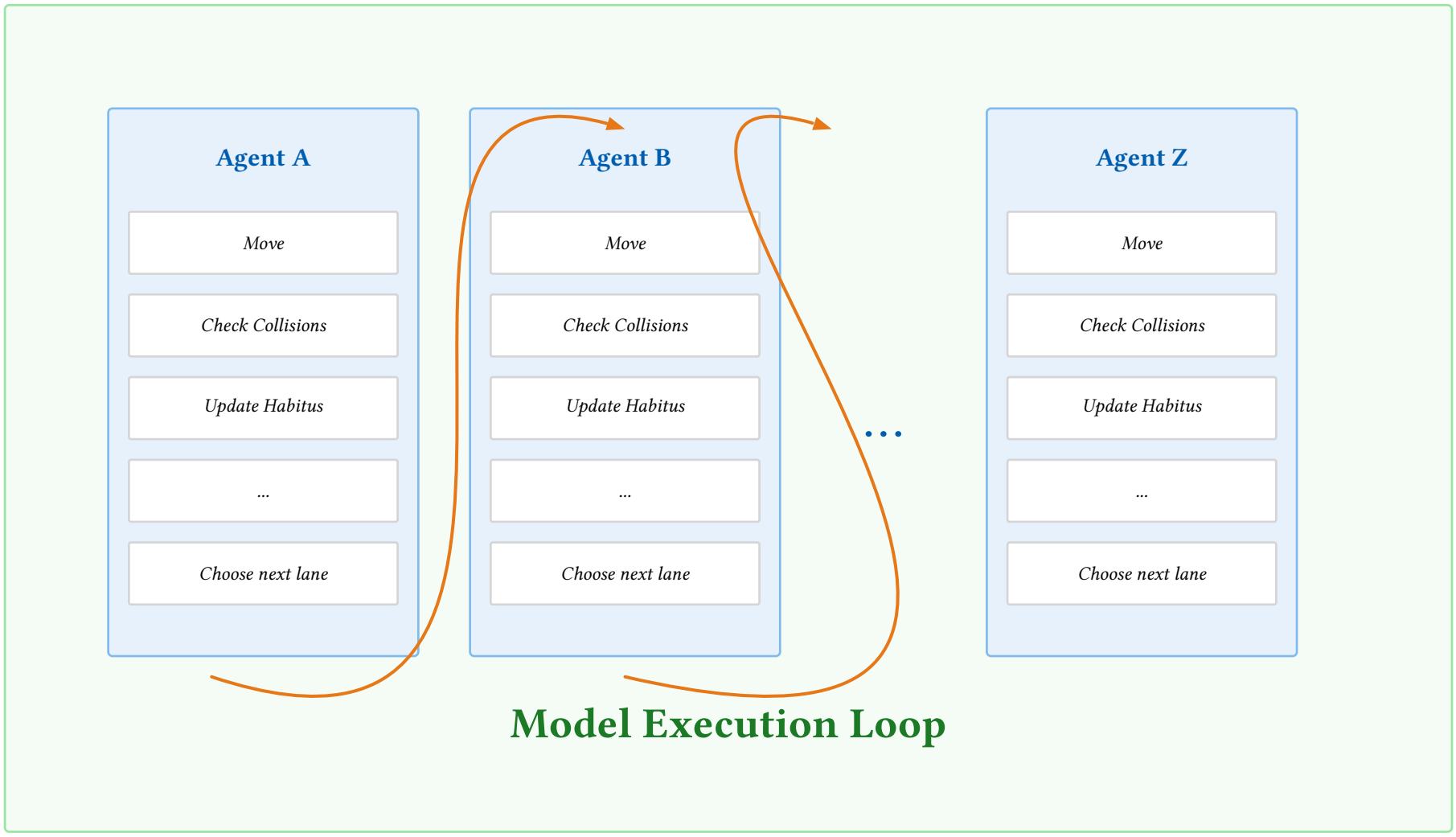
1. Ask why abs enforce sequential logic that is and that we have to go into more detail

3. ABM layouts

3. ABM layouts

—

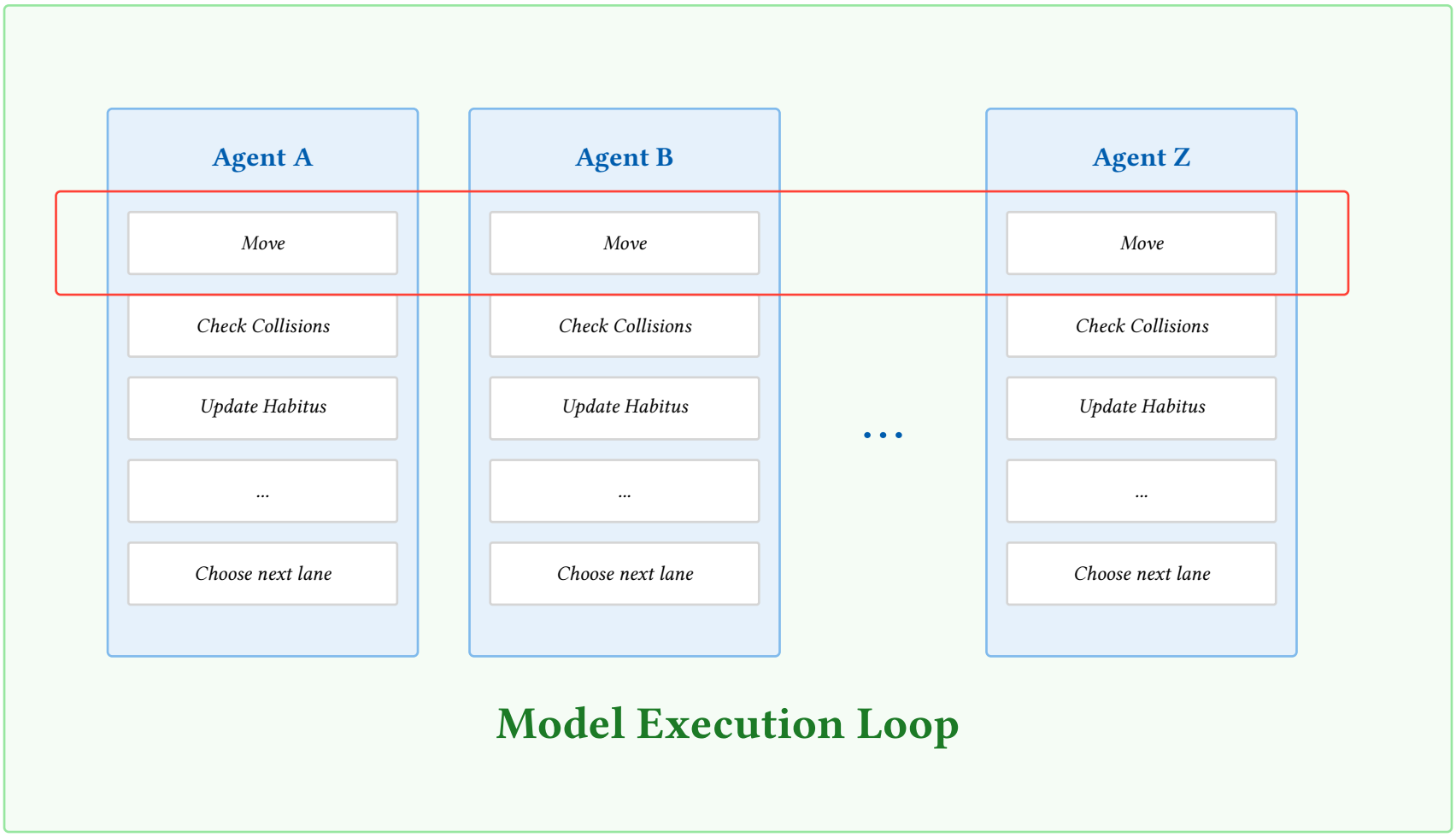
3.1 Traditional ABM layout



3. ABM layouts
— 3.1 Traditional ABM layout

1. Agents have step functions
2. these step functions run sequentially

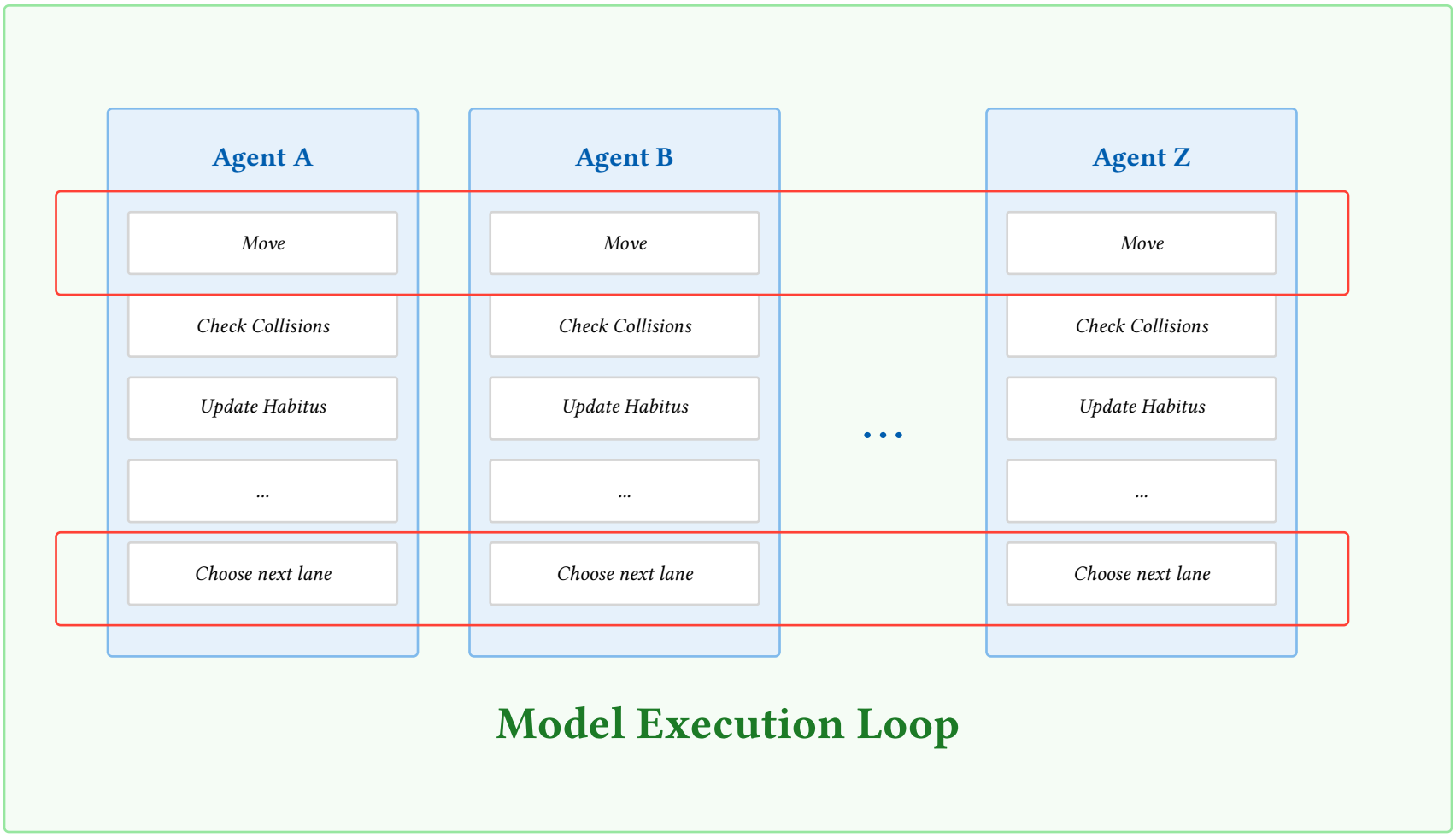
3.2 What if?



3. ABM layouts
— 3.2 What if?

1. What if we group the data not by agents but by systems
2. Luckily paradigm exists => ECS

3.2 What if?



3. ABM layouts
— 3.2 What if?

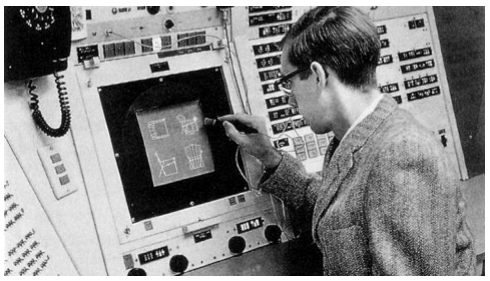
1. What if we group the data not by agents but by systems
2. Luckily paradigm exists => ECS

4. ECS (Entity Component System)!

4. ECS (Entity Component System)!

—

4.1 The history of ECS



- The origins of ECS reach back to 1959, when Ivan Sutherland pioneered a similar idea in a drawing program, one of the first graphical user interfaces (Sutherland, 2003).
- Today, ECS is primarily used in game development, for example in the Bevy engine and Unity DOTS.
- While there is a paper on using ECS for ABMs, it remains a largely unknown programming paradigm in this area (Casals & Brandão, 2025).

4. ECS (Entity Component System)!
— 4.1 The history of ECS

4.2 A short introduction

- ECS stands for **Entity–Component–System**. It is a programming pattern that separates data from behavior.
- **Entities** are unique identifiers. By themselves they do not contain logic or meaning; they simply represent individual objects in the simulation.
- **Components** are small data containers attached to entities. Each component stores one specific aspect of state, such as position, direction, age, or velocity.
- An entity is defined by the set of components it has. For example, a car entity might have components for position, direction, habitus, and step count.

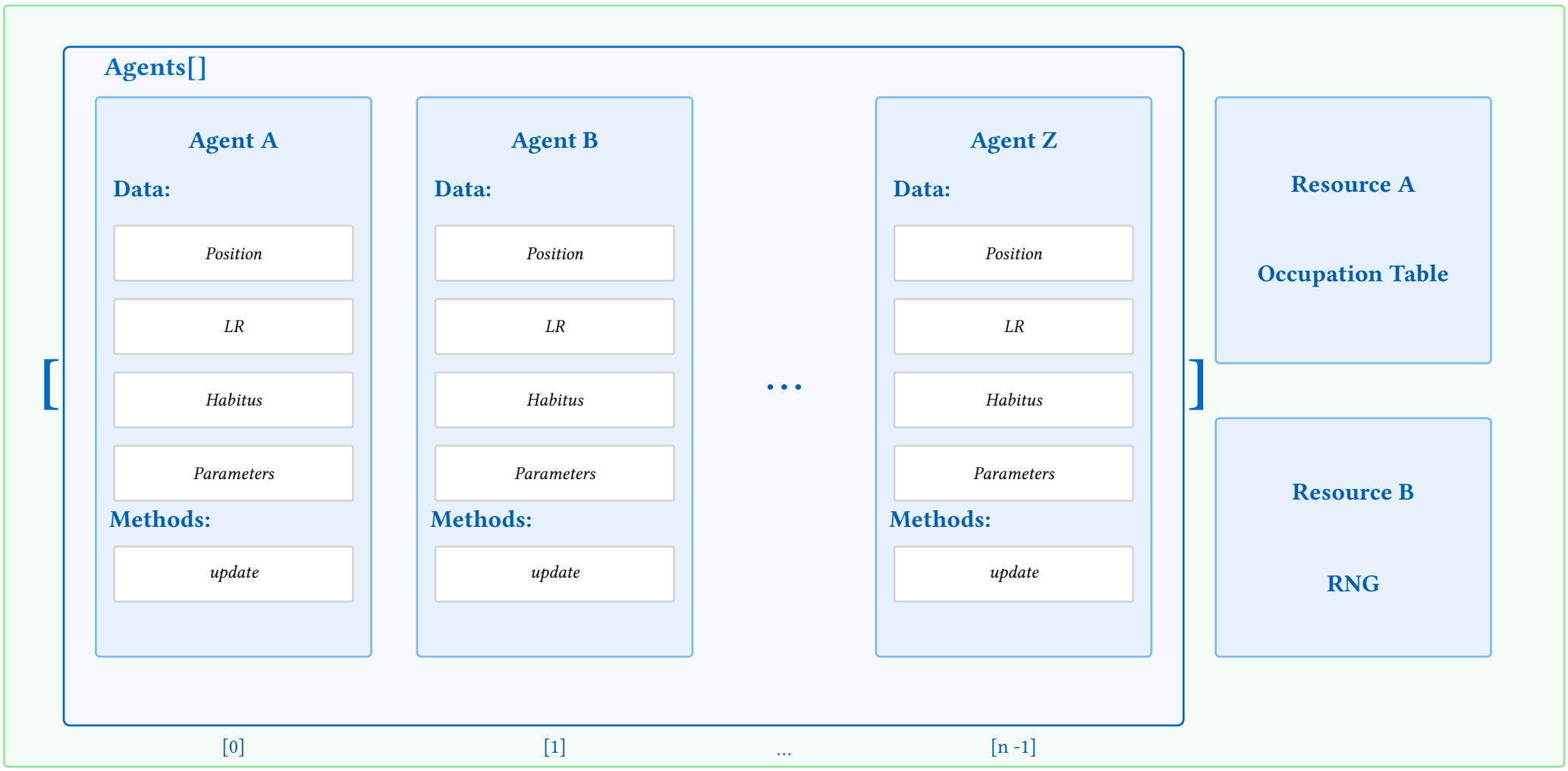
4. ECS (Entity Component System)!
— 4.2 A short introduction

4.2 A short introduction

- **Systems** implement the behavior of the model. They query all entities that have a required set of components and then update those components according to the model rules.
- This means that data and functionality are kept separate: components hold data, while systems contain the logic that acts on that data.

4. ECS (Entity Component System)!
— 4.2 A short introduction

4.3 Data layout in a classic ABM



4.4 ECS data layout

	Entity 1	Entity 2	Entity 3
Component 1	*[Data]*		
Component 2	*[Data]*	*[Data]*	*[Data]*
Component 3		*[Data]*	*[Data]*

4. ECS (Entity Component System)!
— 4.4 ECS data layout

4.5 ABM in ECS terms

Concept	Description	ABM Equivalent	Role in Simulation
Entity	Unique object in the world	Agent	Represents an individual actor in the simulation
Component	Data attached to an entity	Agent attributes / state variables	Stores properties such as position, preferences, or resources
System	Function operating on sets of components	Part of the agent step function	Implements simulation rules and updates state

5. Cars 2

5. Cars 2

—

5.1 Reimplementing the model using ECS

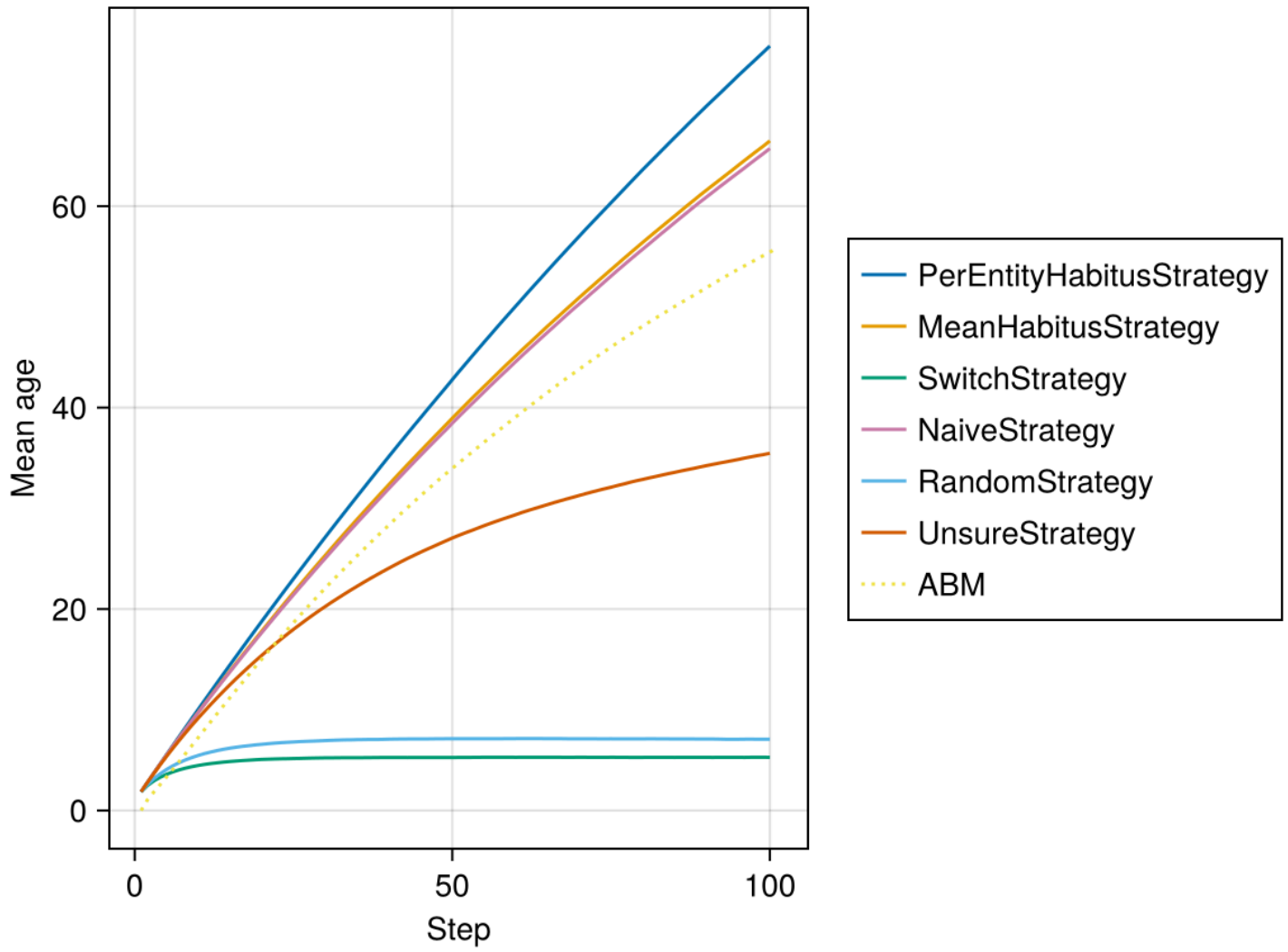
- The natural way to implement the model in ECS is so that cars move simultaneously.
- This leads to a problem:
  - How does a car predict where the other cars are in the next step?
  - --→ different prediction strategies.

5. Cars 2
— 5.1 Reimplementing the model using ECS

5.2 Prediction strategies

Strategy	Prediction
Naive strategy	Other cars just stay on their lane
Switch strategy	Other cars switch their lane
Unsure strategy	Both lanes are equally likely
Per Entity Habitus	The car switches lanes based on its habitus
Mean Absolute Habitus	The car is more likely to stay on a lane the higher the mean absolute habitus is
Random strategy	The car gets assigned a random probability to switch lane

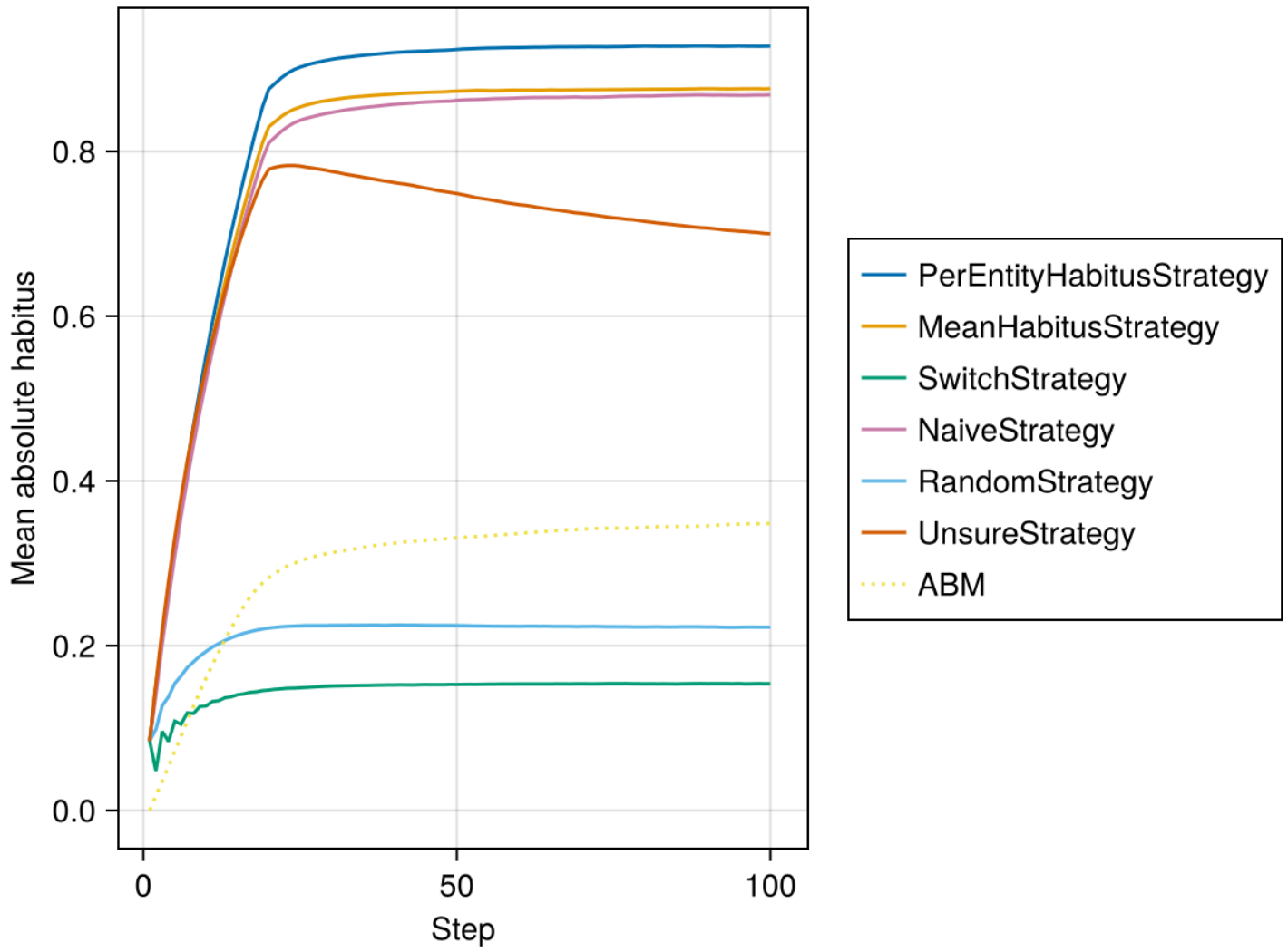
5.3 Which strategy works best



5. Cars 2

— 5.3 Which strategy works best

5.4 The formation of habit



6. Possible benefits and drawbacks of using ECS for ABMs

6. Possible benefits and drawbacks of using ECS for ABMs

6.1 Benefits

- More natural support for parallel updates, both conceptually and computationally
- Potential performance benefits (*JuliaDynamics/ABMFrameworksComparison*, 2026)
- Composition over inheritance
  - Especially useful for heterogeneous agents
- Clear support for modularity and separation of concerns
- Many ECS frameworks support explicit relationships between entities
- Encourages a more systemic rather than purely individual-centered modeling perspective

6. Possible benefits and drawbacks of using ECS for ABMs
— 6.1 Benefits

- Less intuitive than object-oriented programming for many modelers
- Code can become more verbose and structurally complex
- Very limited established literature in the ABM context

- ECS provides a natural way to model simultaneous interaction.
- Reimplementing sequential ABMs in ECS reveals hidden assumptions about prediction and update order.
- In the traffic model, prediction strategy matters greatly for the resulting dynamics.
- ECS is promising for ABMs, but still underexplored both conceptually and methodologically.

7. Appendix

7. Appendix

—

7.1 Stay ratio

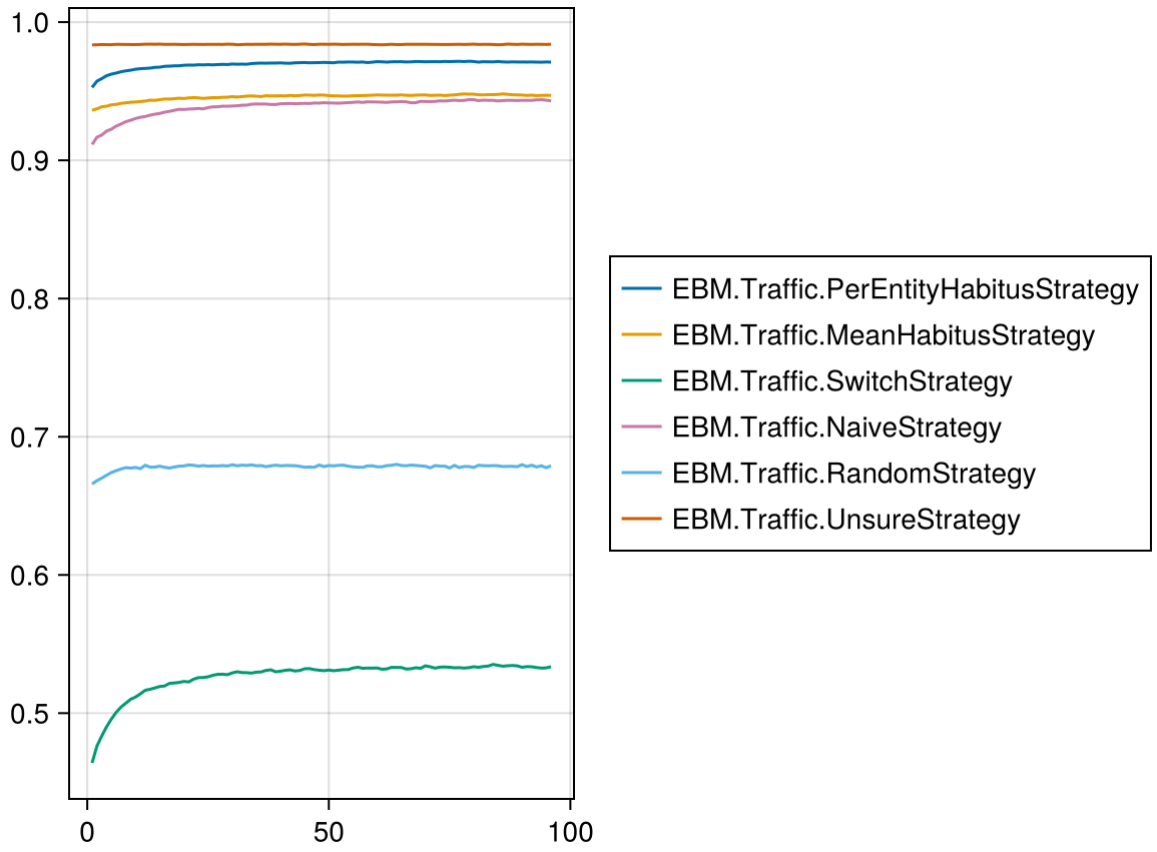


Figure 4: Stay ratio = $\frac{\#\text{ lane switches}}{(\#\text{ total cars}})$

7. Appendix
— 7.1 Stay ratio

7.2 No habit?

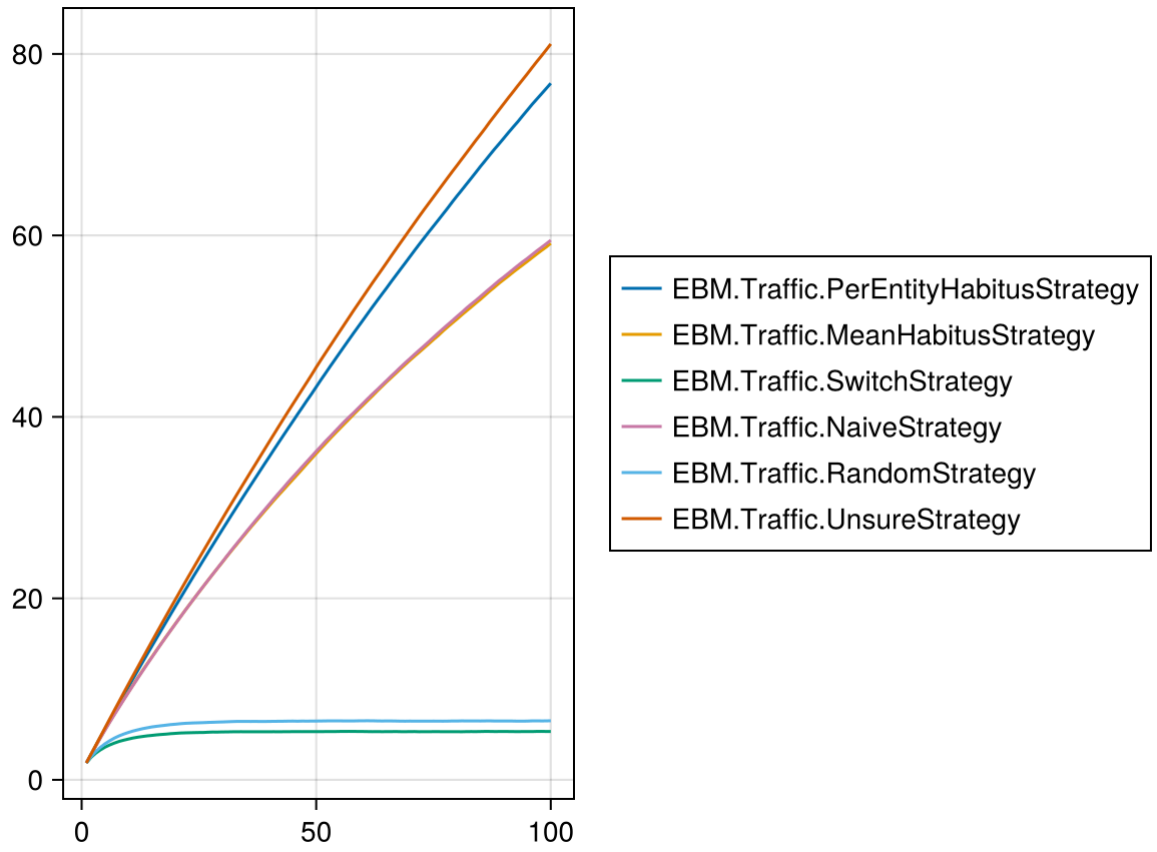


Figure 5: Different strategies when habit is not weighed in in LR calculation

7. Appendix
— 7.2 No habit?

Bibliography

[1] G. M. Hodgson, *Economics in the Shadows of Darwin and Marx: Essays on Institutional and Evolutionary Themes*. Edward Elgar Publishing, 2006. doi: 10.4337/9781781007563.

[2] I. E. Sutherland, “Sketchpad: A Man-Machine Graphical Communication System,” 2003.

[3] A. Casals and A. A. F. Brandão, “HECATE: An ECS-based Framework for Teaching and Developing Multi-Agent Systems.” Accessed: Feb. 02, 2026. [Online]. Available: http://arxiv.org/abs/2509.06431

Bibliography

—

[4] "JuliaDynamics/ABMFrameworksComparison." Accessed: Apr. 06, 2026. [Online]. Available: https://github.com/JuliaDynamics/ABMFrameworksComparison

Bibliography

—